Linear Regression Analysis and Implementation on Housing Dataset

Student Name Shri Mata Vaishno Devi University

Abstract

This project presents a complete implementation of linear regression on a real-world housing dataset. It includes exploratory data analysis (EDA), feature preprocessing, implementation of linear regression using gradient descent and closed-form solution, and comparison with Scikit-learn. The study is extended to multiple linear regression using one-hot encoding and standardization.

1 Introduction

Linear regression is one of the most fundamental supervised learning algorithms used for predicting continuous variables. This project aims to:

- Analyze the housing dataset
- Implement linear regression from scratch
- Compare different approaches (Gradient Descent, Closed Form, Scikit-learn)
- Extend to multiple linear regression

2 Dataset Description

The dataset consists of:

- Total samples: 1460
- Total features: 81
- Target variable: **SalePrice**

The dataset contains both:

- Numerical features (e.g., LotArea, GrLivArea)
- Categorical features (e.g., Neighborhood, HouseStyle)

3 Exploratory Data Analysis (EDA)

3.1 Correlation Analysis

As shown in Figure 1, the feature **GrLivArea** has strong correlation with the target variable **SalePrice**.

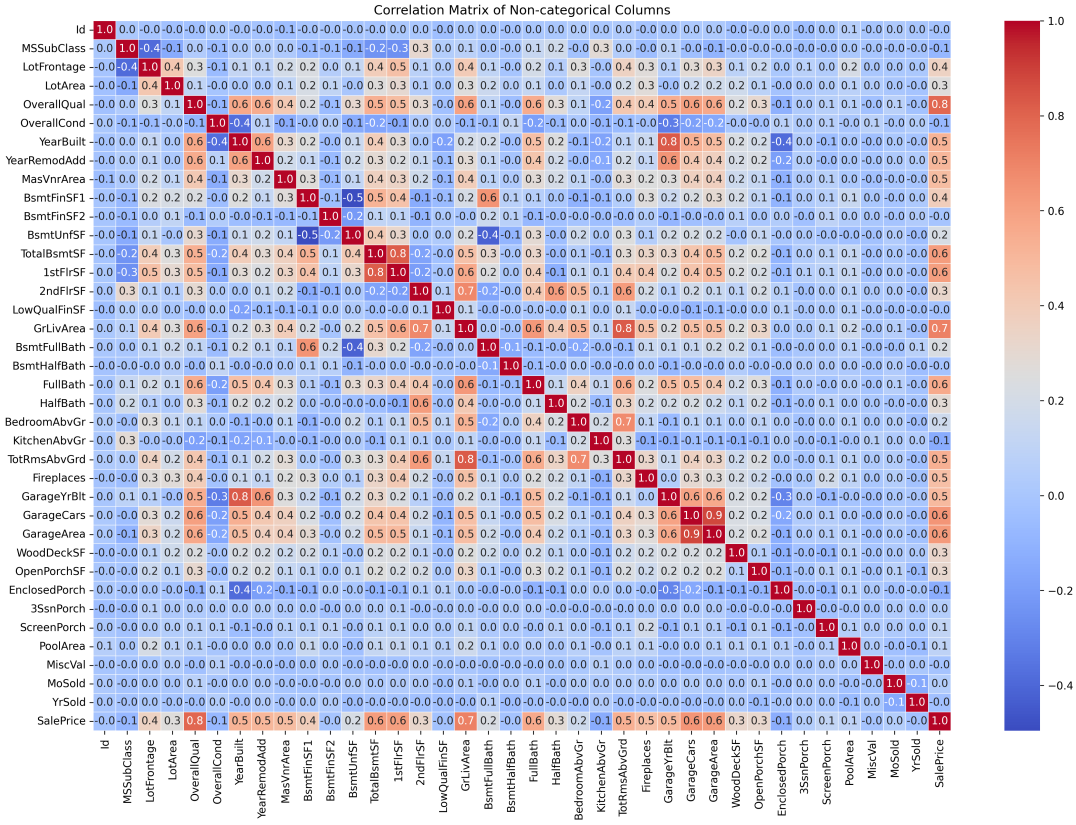


Figure 1: Correlation heatmap showing relationships between numerical features

3.2 Data Inspection

- Used `df.info()` to understand data types and missing values
- Identified categorical and numerical columns

3.3 Numerical Features Analysis

- Box plots were used to detect outliers
- Distribution of numerical features was analyzed

3.4 Categorical Features Analysis

- Bar plots used for frequency distribution
- Helped understand dominant categories

4 Simple Linear Regression

4.1 Feature Selection

The feature **GrLivArea** was selected due to strong correlation with SalePrice.

4.2 Data Preprocessing

- Standardization of features:

$$X_{scaled} = \frac{X - \mu}{\sigma} \tag{1}$$

- Added bias term for intercept

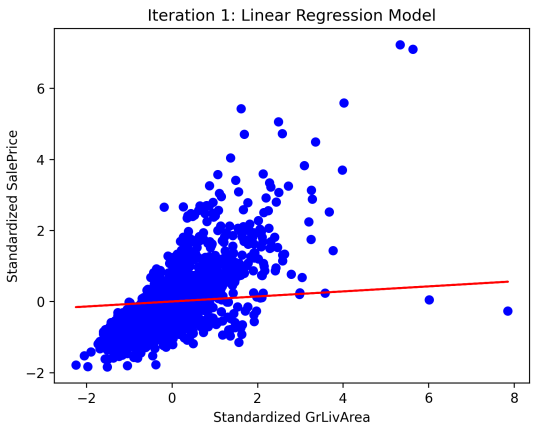
4.3 Model Implementation (Gradient Descent)

The model updates weights using:

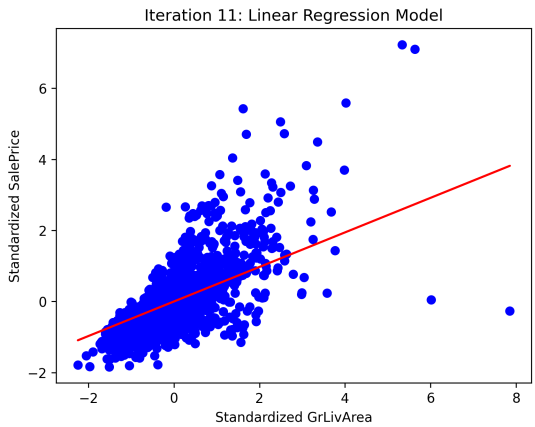
$$w = w - \alpha \nabla L \tag{2}$$

where α is the learning rate.

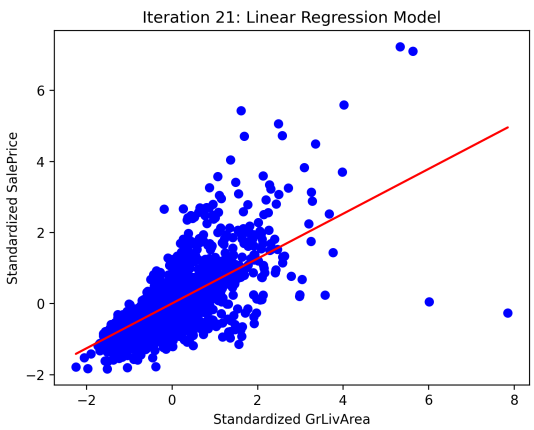
4.4 Observations



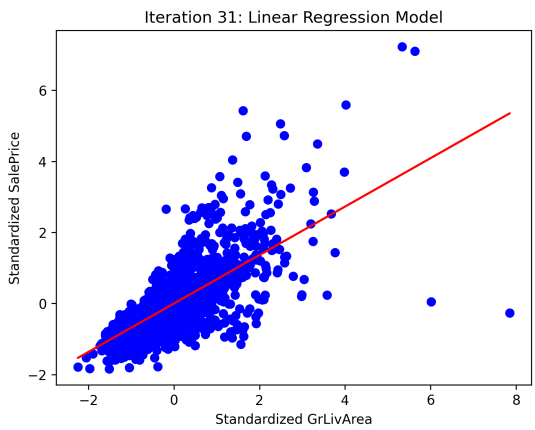
(a) Iteration 1: Initial model with high error



(b) Iteration 11: Model improving



(c) Iteration 21: Model nearing convergence



(d) Iteration 32: Model converged to optimal fit

Figure 2: Model convergence across different iterations

As shown in Figure 2(a), the model initially starts with a poor fit at iteration 1. As gradient descent progresses, the model parameters adjust, leading to improved predictions by iteration 11. Figure 2(c), shows that by iteration 21, the model has nearly converged and closely approximates the optimal regression line.

- Loss decreases with iterations
- Model converges gradually
- Final MSE ≈ 0.49

5 Closed Form Solution

The analytical solution is given by:

$$w = (X^T X)^{-1} X^T y \tag{3}$$

5.1 Observations

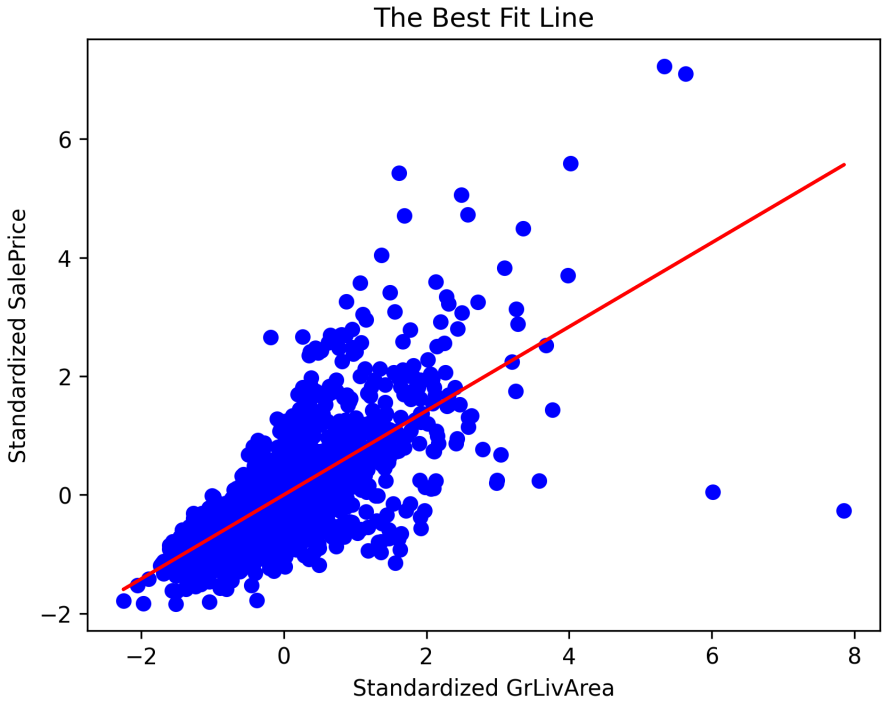


Figure 3: "The result of close form equation"

- Produces similar results as gradient descent
- Faster as no iteration required

6 Scikit-learn Implementation

Using built-in Linear Regression:

- Achieved same MSE as manual implementation
- Confirms correctness of custom implementation

7 Multiple Linear Regression

7.1 Data Preprocessing

- Missing values filled with mean
- One-hot encoding applied to categorical features
- Standardization of numerical features

7.2 Dataset Transformation

- Original features: 81
- After encoding: 246 features

7.3 Train-Test Split

- Training: 80
- Testing: 20

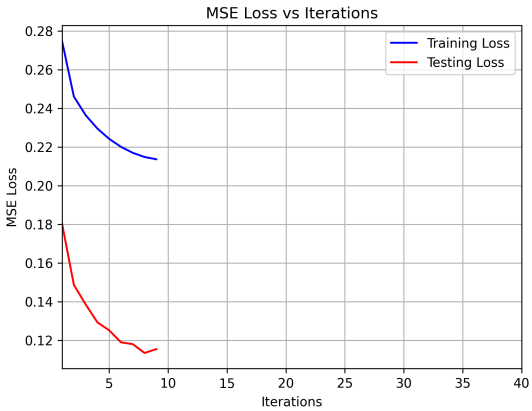
8 Model Training

8.1 Gradient Descent

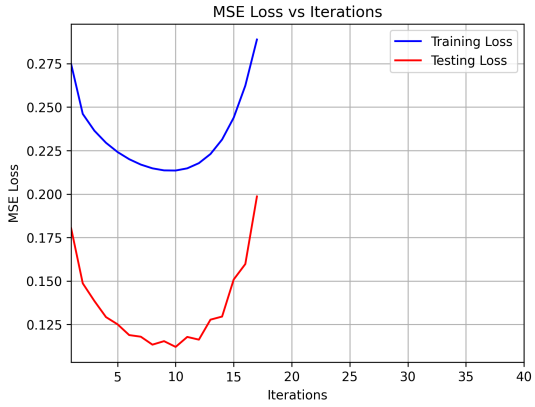
- Model trained for 40 iterations
- Learning rate: 0.1

8.2 Loss Analysis

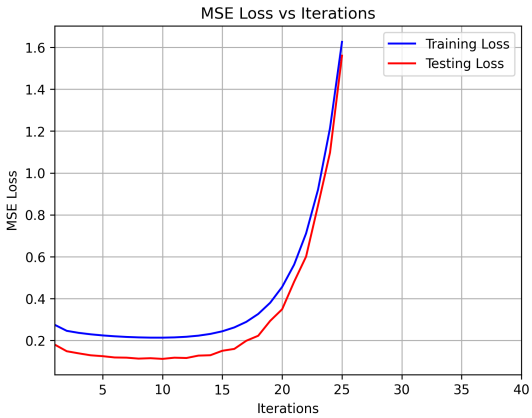
- Training loss decreases steadily but after 10 iteration it starts increasing
- Testing loss follows similar trend
- Indicates proper learning rate is required



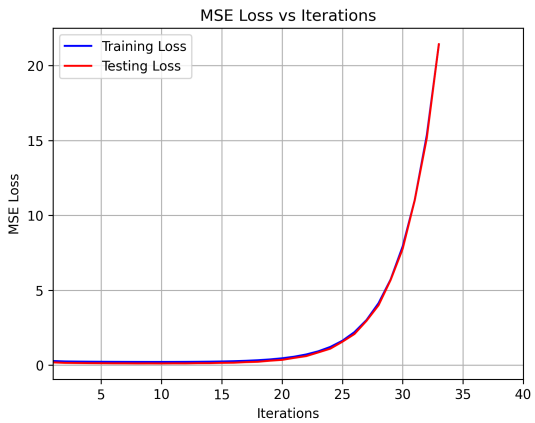
(a) Iteration 1: Initially the loss is decreasing



(b) Iteration 11: Model improving



(c) Iteration 21: Model has started overfitting



(d) Iteration 32: Model is overfited

Figure 4: Model Training and testing loss over different iterations

9 Results

- Simple Linear Regression MSE ≈ 0.49
- Closed-form and Scikit-learn produce similar results
- Multiple Linear Regression improves performance

10 Discussion

- Gradient descent successfully minimizes loss
- Feature scaling is crucial for convergence
- One-hot encoding increases dimensionality

11 Limitations

- No regularization used
- Possible overfitting in high dimensions
- Manual feature engineering limited

12 Future Work

- Add Ridge and Lasso regression
- Use cross-validation
- Perform feature selection
- Try advanced models (Random Forest, XGBoost)

13 Conclusion

This project demonstrates the complete pipeline of linear regression from data exploration to model implementation. The consistency between gradient descent, closed-form, and Scikit-learn validates the correctness of the approach. Extending to multiple regression improves predictive capability, highlighting the importance of feature engineering.